

Architecture and Implementation of a Human-in-the-Loop Sudo Approval Gate for Autonomous LLM Agents

Executive Summary

The rapid integration of autonomous Large Language Model (LLM) agents into production environments necessitates a fundamental reevaluation of operating system security perimeters. Traditionally, automated scripts or unprivileged service accounts requiring elevated permissions rely on binary access controls, most commonly implemented via unrestricted NOPASSWD directives within the `/etc/sudoers` configuration. While highly efficient for deterministic automation, this "all or nothing" authorization model presents unacceptable systemic risk when applied to non-deterministic, probabilistic LLM agents operating within virtualized environments.

The architecture proposed herein replaces this legacy paradigm with a granular, human-in-the-loop (HITL) privilege escalation gate. By intercepting execution attempts directly at the operating system level, suspending the agent process, and delegating the authorization decision to an external human operator, the system achieves a robust balance between agent autonomy and administrative oversight.

This comprehensive technical analysis details the design, implementation, integration, and security hardening of a sophisticated three-tier gating mechanism. The primary tier consists of a custom C-based sudo approval plugin executing within the KubeVirt Virtual Machine (VM). This plugin intercepts execution and establishes synchronous inter-process communication (IPC) over a Unix Domain Socket. The secondary tier is a lightweight local daemon, executing within the same VM, which bridges the Unix socket to a distributed NATS messaging fabric. The tertiary tier encompasses an external FastAPI orchestrator and an Angular 19 frontend, which manage state, enforce auto-approval heuristics, and provide the human operator with the contextual intelligence required to render an informed authorization decision. Through an exhaustive evaluation of the `sudo_plugin(8)` application programming interface (API), daemon concurrency models, real-time frontend notification patterns, and fault-tolerant timeout coordination, this report provides a definitive blueprint for constructing a secure, scalable privilege escalation gate on Ubuntu 24.04 LTS.

Sudo Plugin API Architecture and Implementation

The sudo utility features a highly modular architecture that delegates specific phases of execution to dynamically loaded shared objects. Understanding this architecture is paramount for successfully intercepting and controlling privilege escalation.

Plugin Architecture Evolution and Lifecycle

Prior to version 1.8, sudo was a monolithic binary governed almost exclusively by the `/etc/sudoers` configuration file. Version 1.8 introduced a formalized `sudo_plugin(8)` API, decoupling policy decisions and input/output (I/O) logging from the core front-end.¹ This initial plugin API permitted third-party developers to replace the default sudoers policy engine entirely or to intercept and record the terminal streams of executing commands.¹

Recognizing the need for more granular, layered security controls, version 1.9.0 significantly expanded the API by introducing two new plugin classifications: audit plugins and approval plugins.¹ Audit plugins provide a mechanism for custom logging of both accepted and rejected authentication and execution attempts, granting external tools deep visibility into the execution context.³ Approval plugins, however, are specifically designed to introduce an additional layer of authorization that operates entirely independently of the primary policy engine.¹

The invocation sequence of these plugins follows a strict, deterministic lifecycle managed by the sudo front-end. When an unprivileged user invokes sudo, the front-end first loads and initializes all registered audit plugins.¹ Subsequently, the configured policy plugin (typically the default sudoers plugin) is invoked to determine if the user possesses the baseline authorization to execute the requested command.¹ Only if the policy plugin returns a successful authorization signal are the approval plugins engaged.³ The approval plugins' `open()` and `check()` functions are called sequentially. If, and only if, all configured approval plugins return a success code, the I/O logging plugins are initialized, the process privileges are elevated, and the target command is executed via the `execve` system call.¹

For the autonomous LLM agent environment, this lifecycle provides the exact mechanism required for human-in-the-loop gating. Because the agent-host user currently possesses unrestricted NOPASSWD access within `/etc/sudoers`, the policy plugin will invariably approve the agent's requests.⁶ The custom approval plugin intercepts the workflow immediately after this policy validation, acting as the definitive synchronous barrier before execution.⁵

The Approval Plugin Interface

To integrate with the sudo front-end, a C-based approval plugin must declare and export a specific global data structure of type `approval_plugin`.¹ The sudo dynamic linker searches for this symbol upon loading the shared library.

The `approval_plugin` structure dictates the application binary interface (ABI) and is defined within the `<sudo_plugin.h>` header file.¹ This structure requires the definition of several constant flags and function pointers that handle the plugin's lifecycle and decision logic.

The type field must be explicitly declared as `SUDO_APPROVAL_PLUGIN`, which corresponds to the integer value 4.¹ The version field must be populated with the `SUDO_API_VERSION` macro, allowing the sudo front-end to verify ABI compatibility before attempting to invoke any

function pointers.¹

The functional interface is governed by three primary pointers: open, check, and close. The open function pointer initializes the plugin. It receives the API version, user settings, the user's environment, and any configuration arguments specified in the `/etc/sudo.conf` file.¹ The close function pointer is invoked when the sudo front-end terminates, providing an opportunity to gracefully close file descriptors, terminate socket connections, and free dynamically allocated memory.²

The core authorization logic resides within the check function. The signature for this critical function is defined as follows:

C

```
int (*check)(char * const command_info, char * const run_argv, char * const run_envp, const char **errstr);
```

The arguments passed to the check function contain the entire context of the execution attempt, enabling the plugin to construct a highly detailed authorization request for the human operator.²

The `command_info` argument is a null-terminated array of character pointers, where each string is formatted as a key=value pair containing metadata evaluated by the sudo front-end.¹¹ The plugin must iterate over this array and split each string at the first equals sign (=) to extract the data.¹⁰ Crucial keys within this array include `command` (the absolute, sanitized path to the binary to be executed), `runas_user` (the target user identity), `cwd` (the working directory from which the command was invoked), and `tty` (the terminal device associated with the session).¹¹ The `run_argv` argument contains the exact argument vector that will be passed to the binary, while `run_envp` contains the environment variables that will be injected into the elevated process.²

Return Codes and Acceptance Logic

The integration between the plugin and the sudo front-end relies on a strict set of integer return codes.¹ These return codes dictate the immediate behavior of the privilege escalation workflow.

Return Code	Macro Equivalent	Semantic Meaning and Sudo Behavior
-------------	------------------	------------------------------------

1	sudo.RC.ACCEPT / OK	Accept. The plugin explicitly approves the execution. The sudo front-end proceeds to the next plugin or initiates command execution. ¹
0	sudo.RC.REJECT	Reject. The plugin explicitly denies the execution. The sudo front-end immediately aborts the operation, logs the denial, and exits. ¹
-1	sudo.RC.ERROR	Error. A systemic or environmental error occurred during plugin execution (e.g., memory allocation failure, Unix socket unavailability). The sudo front-end aborts execution. ¹
-2	sudo.RC.USAGE_ERROR	Usage Error. A configuration anomaly was detected. The sudo front-end prints a usage diagnostic message to standard error and aborts execution. ¹

Within the context of the human-in-the-loop gate, the check function will evaluate the response received from the external orchestrator. If the orchestrator signals human approval, the function returns 1. If the human denies the request, or if the auto-deny timeout expires, the function must return 0. If the plugin cannot establish a connection to the local daemon, it must return -1, ensuring the system fails closed and prevents unauthorized escalation.

Unix Socket IPC and Synchronous Blocking Mechanisms

The sudo front-end operates synchronously. When the check function is invoked, the sudo process blocks, suspending the execution of the target command until the plugin returns an integer code.¹ The LLM agent, running in a shell or Python subprocess, remains suspended awaiting the return of the sudo command. This native blocking behavior is the cornerstone of the gating mechanism, eliminating the need for complex OS-level process pausing or signal

handling.

To communicate with the external orchestrator, the C plugin must securely serialize the execution context and transmit it to the local approval daemon. Because the sudo process runs as root and the approval daemon runs as a privileged system service, IPC via a Unix Domain Socket (AF_UNIX) provides the most secure, isolated, and performant transport layer.¹³

The plugin implementation follows a standard socket client pattern.¹³ During the check invocation, the plugin instantiates a stream socket using `socket(AF_UNIX, SOCK_STREAM, 0)`.¹³ It constructs a `sockaddr_un` structure, populating the `sun_path` member with the absolute path to the daemon's listening socket (e.g., `/run/sudo_approval/daemon.sock`), and initiates a connection via the `connect()` system call.¹³

Once connected, the plugin must iterate through `command_info`, `run_argv`, and `run_envp`, serializing the data into a JSON string. Utilizing a lightweight C JSON library, such as `jansson` or `cJSON`, ensures memory-safe string manipulation. The serialized JSON payload is transmitted to the daemon using the `send()` system call.

Crucially, the plugin must not rely on a blocking `recv()` call without an explicit timeout mechanism. If the approval daemon crashes or hangs after accepting the connection, an indefinite blocking `recv()` would permanently freeze the agent's sudo process.¹⁵ To implement a robust wait state, the plugin must utilize the `poll()` or `select()` system call, monitoring the socket file descriptor for `POLLIN` (data available to read).¹⁶

The timeout value passed to `poll()` must exceed the orchestrator's auto-deny timeout (default 300 seconds) to account for network latency and IPC overhead.¹⁵ A recommended timeout of 305,000 milliseconds guarantees that the plugin waits long enough for a legitimate rejection signal from the orchestrator before unilaterally assuming a failure state. If `poll()` times out, the plugin assumes a systemic failure, closes the socket, sets an appropriate error string via the `errstr` pointer, and returns `-1` (`sudo.RC.ERROR`).¹ If `poll()` signals readability, the plugin invokes `recv()`, parses the JSON response to extract the boolean approval flag, and returns `1` or `0` accordingly.

Configuration, Installation, and Execution Context

The C source code must be compiled into a position-independent shared object (`.so`). On Ubuntu 24.04 LTS, utilizing GCC 14, the compilation flags must strictly enforce security and dynamic linking standards¹⁸:

Bash

```
gcc -Wall -Wextra -fPIC -shared -O2 -o approval_gate.so approval_gate.c -lcjson
```

The resulting `approval_gate.so` binary must be installed into the directory designated by the `plugin_dir` parameter, which on Ubuntu 24.04 defaults to `/usr/libexec/sudo/` or `/usr/lib/sudo/`.¹⁹ To prevent the agent-host user from overwriting the binary and bypassing the gate, the file must be owned by `root:root` with strict `0644` permissions.²²

Integration with the `sudo` front-end is achieved by declaring the plugin within `/etc/sudo.conf`.¹⁹ The configuration syntax requires the `Plugin` keyword, the symbol name of the exported struct, the path to the shared object, and an optional list of key-value configuration arguments¹⁹:

```
Plugin approval_gate approval_gate.so socket_path=/run/sudo_approval/daemon.sock
poll_timeout_ms=305000
```

These appended arguments are passed directly into the plugin's `open()` function via the `plugin_options` array, allowing dynamic reconfiguration of the socket path and timeouts without requiring recompilation of the C code.¹¹ Non-comment lines in `/etc/sudo.conf` are strictly parsed, and syntax errors will cause `sudo` to emit a warning and potentially fail to load the plugin.¹⁹

During execution, the plugin code runs entirely within the context of the `sudo` process. Because the `sudo` binary is marked with the `setuid` bit and owned by `root`, the plugin's effective user ID (EUID) is 0 (`root`).²⁵ This elevated execution context ensures the plugin possesses the necessary permissions to connect to restricted Unix Domain Sockets and cannot be preempted or modified by the unprivileged agent-host user initiating the command.

Reference Implementations and Alternatives

Analyzing existing implementations within the `sudo` ecosystem provides architectural validation for the proposed design.

The most prominent open-source reference for privilege gating is `sudo_pair`, developed by Square.²² Designed to enforce dual control for sensitive operations, `sudo_pair` requires a secondary engineer to approve and actively monitor privileged sessions.²² `sudo_pair` utilizes Rust FFI bindings to interact with the C-based `sudo_plugin(8)` API, demonstrating the viability of integrating modern memory-safe languages directly into the `sudo` workflow.²² The project relies heavily on Unix Domain Sockets for IPC between the plugin and a standalone approval script, validating the socket-based architecture chosen for the LLM agent gate.²² While `sudo_pair` intercepts and mirrors terminal I/O streams—a feature unnecessary for non-interactive LLM agents executing discrete shell commands—its handling of the plugin lifecycle and configuration directives serves as an authoritative architectural reference.²⁶

Alternatively, `Sudo` version 1.9 introduced native support for plugins written in Python.¹² This enables rapid prototyping, as demonstrated by the official `BusinessHoursApprovalPlugin` example, which parses the `command_info` tuple and evaluates the current datetime to reject executions outside standard operating hours.²⁸ However, deploying a Python-based plugin

within a security-critical boundary introduces severe vulnerabilities.⁵ A Python plugin relies on the host system's Python interpreter and its vast dependency tree, dramatically expanding the attack surface. Ensuring the absolute immutability of a Python script against an agent that might exploit environment variables (PYTHONPATH), site-packages, or interpreter vulnerabilities is exceptionally difficult.⁵ A compiled, stripped C shared library significantly minimizes the attack surface and is the only secure recommendation for a production environment.

C Implementation Skeleton

The following code provides a functional skeleton illustrating the `approval_plugin` struct, the signature of the check function, and the basic logic required to parse `command_info` and establish the socket connection.

C

```
#include <sudo_plugin.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <poll.h>
#include <string.h>
#include <unistd.h>

/* Global pointers for sudo front-end functions */
static sudo_conv_t plugin_conv;
static sudo_printf_t plugin_printf;

/* Plugin initialization */
static int approval_gate_open(unsigned int version, sudo_conv_t conversation,
                               sudo_printf_t sudo_printf, char * const settings,
                               char * const user_info, int submit_optind,
                               char * const submit_argv, char * const submit_envp,
                               char * const plugin_options, const char **errstr) {
    if (version < SUDO_API_MKVERSION(1, 9)) {
        *errstr = "Plugin requires sudo API version 1.9 or higher.";
        return -1; /* sudo.RC.ERROR */
    }
    plugin_conv = conversation;
    plugin_printf = sudo_printf;
    return 1; /* sudo.RC.ACCEPT */
}
```

```

/* Core authorization logic */
static int approval_gate_check(char * const command_info, char * const run_argv,
                                char * const run_envp, const char **errstr) {

    /* 1. Extract command and cwd from command_info */
    const char *command = NULL;
    const char *cwd = NULL;
    for (char * const *info = command_info; *info!= NULL; info++) {
        if (strncmp(*info, "command=", 8) == 0) {
            command = *info + 8;
        } else if (strncmp(*info, "cwd=", 4) == 0) {
            cwd = *info + 4;
        }
    }

    /* 2. Establish Unix Domain Socket Connection */
    int sock_fd = socket(AF_UNIX, SOCK_STREAM, 0);
    if (sock_fd < 0) {
        *errstr = "Failed to create Unix socket.";
        return -1;
    }

    struct sockaddr_un addr;
    memset(&addr, 0, sizeof(struct sockaddr_un));
    addr.sun_family = AF_UNIX;
    strncpy(addr.sun_path, "/run/sudo_approval/daemon.sock", sizeof(addr.sun_path) - 1);

    if (connect(sock_fd, (struct sockaddr *)&addr, sizeof(struct sockaddr_un)) < 0) {
        *errstr = "Failed to connect to approval daemon. Failing closed.";
        close(sock_fd);
        return -1; /* System failure, block execution */
    }

    /* 3. Transmit JSON Payload (Serialization logic omitted for brevity) */
    /*... send(sock_fd, json_payload, len, 0)... */

    /* 4. Synchronous wait with Timeout via poll() */
    struct pollfd pfd;
    pfd.fd = sock_fd;
    pfd.events = POLLIN;

    /* 305,000 ms timeout to outlast the orchestrator's 300s window */
    int poll_res = poll(&pfd, 1, 305000);

```



```

    if (poll_res == 0) {
        *errstr = "Timeout waiting for orchestrator approval. Failing closed.";
        close(sock_fd);
        return 0; /* sudo.RC.REJECT */
    } else if (poll_res < 0) {
        *errstr = "Poll error on Unix socket.";
        close(sock_fd);
        return -1; /* sudo.RC.ERROR */
    }

    /* 5. Receive and process response */
    char response;
    ssize_t bytes_read = recv(sock_fd, response, sizeof(response) - 1, 0);
    close(sock_fd);

    if (bytes_read > 0) {
        response[bytes_read] = '\0';
        /* Basic string match for demonstration. A robust implementation parses JSON. */
        if (strstr(response, "\"status\": \"APPROVED\"") != NULL) {
            return 1; /* sudo.RC.ACCEPT */
        }
    }

    *errstr = "Execution denied by human operator or invalid response.";
    return 0; /* sudo.RC.REJECT */
}

/* Plugin Teardown */
static void approval_gate_close(void) {
    /* Perform necessary cleanup */
}

/* Export the ABI structure */
sudo_dso_public struct approval_plugin approval_gate = {
    SUDO_APPROVAL_PLUGIN,
    SUDO_API_VERSION,
    approval_gate_open,
    approval_gate_close,
    approval_gate_check,
    NULL /* show_version */
};

```

Approval Daemon Architecture

The approval daemon serves as the critical intermediary component. It must bridge the highly restricted, OS-level Unix Domain Socket interface utilized by the C plugin with the distributed, network-level NATS messaging infrastructure utilized by the FastAPI orchestrator.

Language Selection and Concurrency Model

The architectural decision regarding the implementation language for the daemon centers on a comparison between Go (Golang) and Rust. Both languages are capable of producing statically linked, standalone binaries with minimal runtime dependencies, making them ideal for containerized or virtualized system daemons.³⁰

Architectural Criterion	Go (Golang)	Rust	Assessment for Daemon
Concurrency Model	Goroutines and channels provide a highly abstracted, preemptive scheduling model for handling asynchronous I/O. ³¹	async/await paired with an external executor (e.g., Tokio) offers absolute zero-cost abstractions. ³¹	Go is superior. The daemon must concurrently handle multiple Unix socket connections (if the agent spawns parallel background sudo processes). Go's standard net library natively multiplexes socket I/O over goroutines without the complexity of configuring an async runtime. ³²
Memory Safety and Footprint	Memory-safe via a concurrent garbage collector (GC). Introduces negligible GC pause latencies. ³¹	Absolute memory safety via compile-time borrow checking. Eliminates GC overhead entirely. ³¹	Go is sufficient. While Rust's memory footprint is undeniably smaller ³¹ , the daemon is strictly I/O bound. The microsecond latency introduced by Go's GC is

			irrelevant against a 300-second human approval window.
Ecosystem and NATS Integration	The official nats.go client is the industry standard for NATS integration, featuring highly mature request-reply abstractions. ³⁴	The async-nats crate is robust but requires more extensive boilerplate to manage futures and contexts. ³⁵	Go is superior. The ease of integrating JSON serialization, native Unix Sockets, and the nats.go client drastically accelerates development velocity while maintaining enterprise-grade stability. ³¹

Based on these criteria, **Go** is the recommended language for the approval daemon. Its unparalleled developer ergonomics for networked, highly concurrent I/O applications precisely match the daemon's operational profile.³¹

Unix Socket Server Implementation

The Go daemon must instantiate a Unix Domain Socket listener using the standard library `net.Listen("unix", "/run/sudo_approval/daemon.sock")`. To support multiple concurrent sudo invocations, the daemon must implement an infinite loop that calls `listener.Accept()` and spawns a dedicated goroutine for each incoming connection.¹⁴

Security at the socket layer is paramount. Because the daemon runs as a privileged systemd service, it must explicitly enforce restrictive ownership and permissions on the socket file. Immediately after binding, the daemon must execute `os.Chown()` to set the socket owner to the agent-host user and the group to root, followed by `os.Chmod(0660)`.¹⁴ This ensures that only the unprivileged agent user (to initiate the connection) and the root user (executing the sudo plugin) can interface with the daemon, preventing unauthorized local processes from injecting approval requests.

NATS Messaging: The Request-Reply Pattern

While the daemon could theoretically communicate with the orchestrator via HTTP REST calls, leveraging the existing NATS infrastructure deployed on the Harvester cluster provides superior resilience, dynamic routing, and lower latency.

The architecture must utilize the NATS **Request-Reply** pattern. Unlike asynchronous Publish-Subscribe (Pub/Sub), the Request-Reply pattern provides a synchronous-style RPC interface over the asynchronous messaging backbone.³⁴

When the Go daemon receives a JSON payload from the C plugin over the Unix socket, it executes the following sequence:

1. **Request Construction:** The daemon injects the unique `job_id` and the KubeVirt `agent_id` into the JSON payload alongside the execution context.
2. **NATS Invocation:** The daemon invokes the `Request()` method of the `nats.go` client.³⁴ For example:

```
Go  
msg, err := nc.Request(fmt.Sprintf("agent.vm.%s.sudo.request", jobID), payload,  
300*time.Second)
```
3. **Inbox Management:** Under the hood, the NATS client dynamically generates a unique, ephemeral reply subject (an `_INBOX`) and attaches it to the outbound message.³⁴
4. **Routing:** The message is routed via the NATS Leaf Node on the agent cluster to the Hub Node on the orchestrator cluster.³⁴
5. **Orchestrator Processing:** The FastAPI orchestrator, subscribed to `agent.vm.*.sudo.request`, receives the message, parses the payload, and notes the ephemeral `_INBOX` subject. It records the state in PostgreSQL and waits for the Angular UI to push a decision.³⁴
6. **Reply Routing:** Once the human operator decides, the orchestrator publishes the decision directly to the ephemeral `_INBOX` subject.³⁴
7. **Resolution:** The Go daemon's blocking `nc.Request()` call resolves. The daemon parses the response and writes the boolean decision back to the Unix socket, unblocking the C plugin.³⁴

This pattern elegantly handles scaling and transient network failures. If the orchestrator restarts, the pending request remains active. So long as the NATS server maintains the connection and the 300-second timeout has not elapsed, the orchestrator can process the request and route the reply back to the specific daemon instance waiting on the `_INBOX`.³⁸

Systemd Integration and Security Hardening

To ensure the daemon is highly available and tightly integrated with the VM's lifecycle, it must be managed as a `systemd` service. Go applications can natively interact with `systemd` via the `coreos/go-systemd/daemon` package. By setting the service `Type=notify`, the Go daemon can execute `daemon.SdNotify(false, daemon.SdNotifyReady)` immediately after successfully binding the Unix socket and establishing the NATS connection.⁴⁰ This guarantees that the management-daemon (which oversees the VM's overall state) knows the environment is healthy only when the gating mechanism is actively listening.⁴⁰

Furthermore, the `systemd.service` file must enforce strict security boundaries to prevent the daemon from being exploited to bypass the gate⁴²:

- **ProtectSystem=strict:** Mounts the entire OS filesystem as read-only for the daemon,

prohibiting unauthorized file modifications.⁴³

- **ProtectHome=read-only:** Revokes write access to all user home directories, neutralizing attempts to alter `.bashrc` or SSH keys.⁴³
- **PrivateTmp=yes:** Isolates the daemon's `/tmp` and `/var/tmp` directories, neutralizing symlink attacks and temporary file prediction exploits.⁴³
- **NoNewPrivileges=yes:** Prevents the daemon and any child processes from elevating privileges via `setuid` or `setgid` executables, ensuring the daemon remains confined to its original execution context.⁴²
- **CapabilityBoundingSet=:** Strips all Linux kernel capabilities from the process, leaving it with the absolute minimum required to perform network I/O.

Rate Limiting and DoS Mitigation

Autonomous agents caught in logic loops can easily trigger rapid, repetitive failures. If an agent rapidly attempts `sudo` commands, it could theoretically flood the NATS broker, overwhelm the FastAPI orchestrator, and inundate the human operator with approval requests.

To mitigate this Denial of Service (DoS) vector, the Go daemon must implement rate limiting locally. A token bucket algorithm, readily available via the golang.org/x/time/rate package, is the optimal approach. The token bucket permits a small initial burst of requests (e.g., 3 requests to allow for complex initialization scripts) followed by a strict sustained limitation (e.g., 1 request per 30 seconds).

If an incoming connection on the Unix socket violates the rate limit, the Go daemon bypasses NATS entirely. It immediately writes a JSON rejection payload to the socket with a specific error message (e.g., "Rate limit exceeded") and terminates the connection. The C plugin receives this rejection and returns 0 (`sudo.RC.REJECT`), instantly failing the agent's command without generating upstream network traffic or UI noise.

Orchestrator and User Interface Integration

The FastAPI orchestrator acts as the authoritative state machine, bridging the NATS messaging layer with the Angular 19 frontend to facilitate the human-in-the-loop workflow.

PostgreSQL Schema and Auditability

Tracking the state of privilege escalation requests is critical for operational debugging and security auditing.⁴⁶ The PostgreSQL schema must securely link the approval request to the existing job context and provide a cryptographically sound audit trail.⁴⁶

Column	Data Type	Description and Constraints
--------	-----------	-----------------------------

id	UUID	Primary Key. Uniquely identifies the request.
job_id	UUID	Foreign Key referencing the originating LangGraph job. ⁴⁶ Indexed for rapid filtering in the Cockpit UI.
agent_id	UUID	Foreign Key referencing the specific KubeVirt VM.
command_payload	JSONB	Stores the execution context (command, arguments, environment). JSONB allows the FastAPI application to efficiently query against specific command structures using GIN indexes. ⁴⁸
status	VARCHAR(20)	Restrictive Enum: PENDING, APPROVED, DENIED, EXPIRED, AUTO_APPROVED.
requested_at	TIMESTAMPTZ	The precise UTC timestamp when the request was intercepted and stored. ⁴⁹
resolved_at	TIMESTAMPTZ	The timestamp of the human decision or automatic expiration. Nullable until resolution.
resolved_by	UUID	Foreign Key referencing the specific human operator who issued the decision. Nullable if expired or auto-approved. ⁴⁶

This schema guarantees a strict, non-repudiable audit log. Security teams can query this table to reconstruct exactly which operator approved a destructive command, satisfying stringent

compliance requirements.⁴⁶

Real-Time Notification: SSE vs. WebSockets

When the FastAPI orchestrator transitions a new request into the PENDING state, it must immediately alert the human operator via the Angular 19 Cockpit UI. The architectural decision lies between leveraging the existing WebSocket infrastructure or implementing Server-Sent Events (SSE).

Evaluation Metric	WebSockets	Server-Sent Events (SSE)	Application to Approval Gate
Communication Flow	Bidirectional, full-duplex communication. ⁵⁰	Unidirectional, server-to-client push. ⁵⁰	SSE is optimal. The server only needs to push the notification of the pending request. The client's response (Approve/Deny) is a discrete, standard HTTP POST request containing the authorization token. ⁵²
Protocol and Infrastructure	Custom ws:// or wss:// protocol requiring HTTP upgrade headers. ⁵¹	Native HTTP/1.1 and HTTP/2 transport. ⁵¹	SSE is optimal. Operates seamlessly over standard HTTP/2, automatically benefiting from connection multiplexing without requiring specialized load balancer configurations. ⁵¹
Connection Resilience	Requires complex, manually implemented	Native browser support via the EventSource API,	SSE is optimal. Drastically simplifies the

	ping/pong heartbeats and exponential backoff reconnection logic. ⁵¹	featuring automatic reconnection and Last-Event-ID tracking. ⁵¹	Angular client-side implementation and guarantees message delivery even during brief network interruptions. ⁵²
Payload Formatting	Supports both UTF-8 Text and Binary frames. ⁵¹	Restricted to UTF-8 Text payloads. ⁵¹	SSE is optimal. The payload consists exclusively of structured JSON string data representing the command context. ⁵¹

Given the strictly unidirectional requirement of the notification and the immense operational simplicity of the EventSource API, **SSE is the unequivocally superior architectural choice** for broadcasting approval requests.⁵² The existing WebSocket infrastructure can remain dedicated exclusively to the high-throughput builder chat functionality.

Angular 19 Signals and Zone-less Reactivity

Angular 19 promotes a modern, fine-grained reactivity model centered around Signals.⁵⁶ Integrating the asynchronous SSE data stream into this paradigm requires specific architectural patterns to ensure performance in a zone-less application.

Within an Angular Service, the EventSource connection is established, pointing to the FastAPI notification endpoint. Incoming JSON payloads are parsed and appended to a private WritableSignal<ApprovalRequest>. To encapsulate state management and prevent component-level mutation, the service exposes this data to the UI components via the asReadOnly() method.⁵⁸

UI components consume this readonly Signal, utilizing computed() signals to dynamically derive specific UI states, such as the count of pending requests or the presence of high-risk commands.⁵⁸ Crucially, because the EventSource.onmessage callback fires outside of Angular's standard detection zones, the service must explicitly invoke ApplicationRef.tick() upon updating the WritableSignal. This guarantees that the DOM repaints instantly, ensuring the human operator sees the approval request without delay while avoiding the systemic overhead of zone.js.⁵⁶

Approval UX and Auto-Approval Rule Engine

The User Experience (UX) of the approval gate must mitigate operator fatigue while maximizing

situational awareness. Drawing from best practices in mature CI/CD systems like Terraform Cloud or ArgoCD, the Cockpit UI should present the request clearly, differentiating the binary path from its arguments.⁶⁰ High-risk flags (e.g., `rm -rf`, `chmod 777`, `chown`) should be parsed and highlighted in red. The UI must display a prominent countdown timer representing the 300-second expiration window.

To further reduce cognitive load and prevent operators from blindly approving repetitive, low-risk requests, the FastAPI orchestrator must implement an Auto-Approval Rule Engine. These rules, stored within a dedicated PostgreSQL table, define safe operational bounds.

When a request arrives, the orchestrator evaluates the `command_payload` against the active ruleset. The evaluation engine utilizes strictly bounded regular expressions to perform pattern matching (e.g., `^apt-get install [a-zA-Z0-9\-\+]+$`). To prevent Regular Expression Denial of Service (ReDoS) attacks initiated by a malicious agent generating pathological string inputs, the evaluation engine must execute within a strict, sub-second timeout wrapper. If a command perfectly matches an auto-approval rule, the orchestrator immediately publishes an APPROVED payload to the NATS `_INBOX`, logs the transaction as `AUTO_APPROVED` in PostgreSQL, and bypasses the Angular UI entirely. This allows the agent to install dependencies or perform routine maintenance unhindered, while reserving the human gate for complex or potentially destructive operations.

Security Analysis and Implementation Considerations

The introduction of an IPC bridge and a network-spanning messaging layer into a privilege escalation pathway significantly expands the system's attack surface. Thorough threat modeling is required to ensure the integrity of the gate.

Attack Surface and Mitigation Strategies

1. Plugin Binary Substitution and Configuration Tampering: The most direct attack vector involves the unprivileged agent-host user discovering a local exploit to overwrite the `approval_gate.so` binary or modify `/etc/sudo.conf` to remove the plugin directive.²² *Mitigation:* The foundational security relies on strict file permissions. Both the compiled shared library and the configuration file must be owned by `root:root` with read-only permissions (`0644`) for other users.²² Furthermore, during the KubeVirt VM provisioning process via Packer, the `chattr +i` (immutable) attribute should be applied to both files. This ensures that even if the agent discovers a privilege escalation vulnerability unrelated to `sudo`, it cannot tamper with the gating mechanism without first bypassing the immutable bit, adding a significant layer of defense-in-depth.

2. Unix Socket Spoofing and Flooding: An intelligent agent might deduce the presence of the `/run/sudo_approval/daemon.sock` file and attempt to interface with it directly, either to spoof a fake approval response or to flood the daemon with garbage data, triggering a Denial of Service.¹⁴ *Mitigation:* As previously established, socket permissions must be restricted to `0660` with ownership `agent-host:root`. While this allows the agent to open a connection, the Go

daemon enforces strict JSON schema validation. Any malformed payload results in immediate connection termination. Furthermore, the token-bucket rate limiter neutralizes flooding attempts, ensuring the daemon's CPU and NATS connection remain uncompromised. Spoofing an approval response is impossible, as the agent cannot inject data into the specific file descriptor opened by the highly privileged sudo process.

3. Environment Injection and Path Traversal: The agent might attempt to subvert the approval UI by altering the \$PATH environment variable or defining bash aliases to disguise malicious commands as benign ones (e.g., aliasing ls to rm -rf /).⁶² *Mitigation:* The sudo front-end natively sanitizes environment variables and resolves absolute binary paths before invoking the plugin. The C plugin must strictly rely on the absolute path provided within the command_info array, explicitly ignoring the user's raw input.¹¹ The orchestrator UX displays this resolved absolute path, ensuring the human operator reviews the definitive execution target.

Cross-Component Timeout Coordination

The architecture relies on four discrete boundaries, each possessing its own timeout mechanism.⁶ Misalignment of these timeouts guarantees race conditions, potentially resulting in the orchestrator recording an "APPROVED" state long after the sudo plugin has timed out and failed the command.

To guarantee systemic consistency, the timeout hierarchy must cascade backward from the Orchestrator to the C plugin, with each lower layer granting a slightly longer grace period to accommodate network latency and processing overhead:

Component	Timeout Value	Expiration Behavior
FastAPI Orchestrator	295 Seconds	The system marks the request as EXPIRED in PostgreSQL. An explicit REJECT payload is published via NATS to the ephemeral_INBOX.
NATS Client (Go Daemon)	300 Seconds	The nc.Request() call times out. ³⁴ The daemon generates a local REJECT payload and writes it to the Unix socket.
Unix Socket (C Plugin)	305 Seconds	The poll() system call times out. ¹⁵ The plugin assumes

		total systemic failure, closes the file descriptor, and returns -1 (sudo.RC.ERROR).
--	--	---

Notably absent from this hierarchy is the standard sudo timestamp_timeout (password caching).¹⁷ Because the agent-host user is explicitly configured with NOPASSWD in /etc/sudoers⁷, password caching is irrelevant.¹⁷ The approval plugin functions as the sole temporal gate.

Ubuntu 24.04 LTS Specifics

The deployment target consists of KubeVirt VirtualMachines based on Ubuntu 24.04 LTS (Noble Numbat), constructed via HashiCorp Packer.¹⁸

Sudo Version and Capabilities

Ubuntu 24.04 LTS ships with sudo version 1.9.15p5, ensuring complete compatibility with the SUDO_APPROVAL_PLUGIN API introduced in version 1.9.0.² This specific version includes critical backported security patches addressing recent vulnerabilities such as CVE-2025-32462 (host option privilege escalation) and CVE-2025-32463 (chroot execution bypass).⁶³

It is important to note that Canonical has initiated a transition toward replacing the C-based sudo.ws implementation with the Rust-based sudo-rs project, targeting Ubuntu 25.10 (Questing Quokka) as the default integration point.⁶⁷ However, Ubuntu 24.04 retains the traditional C implementation by default.⁶⁷ This guarantees that standard C Foreign Function Interface (FFI) conventions, memory management semantics, and dynamic linker behaviors operate perfectly without encountering potential compatibility layer abstractions introduced by sudo-rs.⁶⁷

Toolchain and Paths

The plugin must be compiled utilizing the default GCC 14 toolchain provided by Ubuntu 24.04.¹⁸ The standard installation path for compiled sudo plugins on this OS release is /usr/libexec/sudo/ or /usr/lib/sudo/.¹⁹ The Go daemon can leverage the updated systemd v255.4 features natively integrated into the OS.¹⁸

Recommended Build Order and Testing Strategy

Given the complex, asynchronous nature of a network-bridged IPC architecture, a simultaneous "big bang" deployment will severely hinder debugging efforts. The implementation should adhere to a strict, phased integration sequence.

Phase 1: Daemon Isolation and NATS Mocking

Development commences with the Go daemon. A simple Python script is utilized to generate JSON payloads and push them to the Unix socket, mimicking the behavior of the C plugin.

Simultaneously, the nats req command-line tool, or a lightweight Python script utilizing nats-py, is used to simulate the FastAPI orchestrator. This isolates the Go daemon, proving that it correctly parses incoming Unix socket data, successfully wraps it into a NATS Request, handles the dynamic _INBOX routing, and returns the mock decision back down the socket stream.

Phase 2: C Plugin Compilation and Local IPC Validation The C approval plugin is developed and compiled into a shared object. The Go daemon is temporarily replaced with a basic socketserver or socat listener that indiscriminately returns a hardcoded 1 (Accept) or 0 (Reject) JSON payload. The plugin is registered in /etc/sudo.conf.²¹ Testing involves executing arbitrary commands as the agent-host user and validating that the sudo process successfully suspends execution, connects to the socket, and correctly interprets the mock responses to either allow or deny the command. System logs (/var/log/auth.log) are monitored to verify the plugin's return codes.

Phase 3: FastAPI Orchestrator and Database Integration

The PostgreSQL schema is deployed. The FastAPI application is constructed to integrate the nats-py client and expose the necessary HTTP endpoints. A generic REST client (e.g., Postman or curl) is utilized to simulate incoming requests from the NATS broker. The focus here is verifying state transitions within PostgreSQL, testing the efficiency of the JSONB queries, and validating the performance and safety bounds of the Regex Auto-Approval Rule Engine.

Phase 4: Angular 19 UI and SSE Implementation The Cockpit frontend is developed. The Angular Service is implemented to consume the FastAPI Server-Sent Events endpoint, mapping the incoming stream to a WritableSignal.⁵⁸ The UI components are constructed to reactively render the pending requests. Testing ensures that ApplicationRef.tick() functions correctly in the zone-less environment⁵⁶ and that the Approve/Deny buttons correctly trigger POST requests back to the orchestrator, completing the human interaction loop.

Phase 5: End-to-End Environment Validation

The final phase involves deploying the compiled C plugin, the Go daemon, and the configured /etc/sudo.conf file into the Ubuntu 24.04 base image via Packer. A KubeVirt VirtualMachine is provisioned on the Harvester cluster. The LangGraph agent is instructed to execute a complex, privileged command via its run_command tool. The entire pipeline is observed to ensure the agent is halted, the request propagates via NATS to the orchestrator, the Angular UI renders the context, the human operator issues an approval, and the agent resumes execution seamlessly. Extensive testing of the cascading timeout hierarchy is performed by deliberately delaying human input to ensure the system consistently fails closed without introducing hanging processes or zombie VMs.

Works cited

1. sudo_plugin(8) - Linux manual page - Michael Kerrisk, accessed March 16, 2026, https://man7.org/linux/man-pages/man8/sudo_plugin.8.html
2. sudo_plugin - man pages section 8: System Administration Commands, accessed

March 16, 2026,

https://docs.oracle.com/cd/E88353_01/html/E72487/sudo-plugin-8.html

3. What's new in Sudo 1.9 | 18x, accessed March 16, 2026,
<https://www.socallinuxexpo.org/scale/18x/presentations/whats-new-sudo-19/>
4. What's New in the Sudo 1.9 Update - Privileged Access Management - Blogs - One Identity, accessed March 16, 2026,
<https://www.oneidentity.com/community/blogs/b/privileged-access-management/posts/whats-new-in-sudo-update>
5. 5 new sudo features you need to know in 2020 | Opensource.com, accessed March 16, 2026, <https://opensource.com/article/20/10/sudo-19>
6. Security implications of increasing sudo timeout in terminal, accessed March 16, 2026,
<https://security.stackexchange.com/questions/276430/security-implications-of-increasing-sudo-timeout-in-terminal>
7. NOPASSWD Sudo on Ubuntu 24.04 not working, accessed March 16, 2026,
<https://askubuntu.com/questions/1522123/nopasswd-sudo-on-ubuntu-24-04-not-working>
8. regular expression - Sudo regex 1.9.16p2 - not working - Unix & Linux Stack Exchange, accessed March 16, 2026,
<https://unix.stackexchange.com/questions/793996/sudo-regex-1-9-16p2-not-working>
9. sudo_pair/sudo_plugin-sys/include/sudo_plugin.h at master · square/sudo_pair - GitHub, accessed March 16, 2026,
https://github.com/square/sudo_pair/blob/master/sudo_plugin-sys/include/sudo_plugin.h
10. sudo_plugin(5) - Linux manual page - man7.org, accessed March 16, 2026,
https://man7.org/linux/man-pages/man5/sudo_plugin.5.html
11. sudo_plugin — Sudo Plugin API - Ubuntu Manpage, accessed March 16, 2026,
https://manpages.ubuntu.com/manpages/xenial/man8/sudo_plugin.8.html
12. sudo_plugin_python(5) — sudo-plugin-python - openSUSE Manpages Server, accessed March 16, 2026,
https://manpages.opensuse.org/Tumbleweed/sudo-plugin-python/sudo_plugin_python.5
13. UNIX domain socket tutorial, accessed March 16, 2026,
https://systemprogrammingatntu.github.io/mp2/unix_socket.html
14. Brushing Up My C. Building A Unix Domain Socket Client/Server (Part I) - DEV Community, accessed March 16, 2026,
https://dev.to/nikos_katsanos/brushing-up-my-c-building-a-unix-domain-socket-client-server-part-i-324g
15. Block Socket with Unix and C/C++ Help - Stack Overflow, accessed March 16, 2026,
<https://stackoverflow.com/questions/5494052/block-socket-with-unix-and-c-c-help>
16. Problem with blocking unix domain sockets - c++ - Stack Overflow, accessed March 16, 2026,

- <https://stackoverflow.com/questions/2675648/problem-with-blocking-unix-domain-sockets>
17. How to make super-user permissions last longer than using sudo? - Ask Ubuntu, accessed March 16, 2026,
<https://askubuntu.com/questions/671026/how-to-make-super-user-permissions-last-longer-than-using-sudo>
 18. Ubuntu 24.04 LTS (Noble Numbat) Release Notes, accessed March 16, 2026,
<https://discourse.ubuntu.com/t/ubuntu-24-04-lts-noble-numbat-release-notes/39890>
 19. sudo.conf — configuration for sudo front-end - Ubuntu Manpage, accessed March 16, 2026,
<https://manpages.ubuntu.com/manpages/jammy/man5/sudo.conf.5.html>
 20. sudoers — default sudo security policy plugin - Ubuntu Manpage, accessed March 16, 2026,
<https://manpages.ubuntu.com/manpages/xenial/man5/sudoers.5.html>
 21. sudo.conf(5) - Linux manual page - Michael Kerrisk, accessed March 16, 2026,
<https://man7.org/linux/man-pages/man5/sudo.conf.5.html>
 22. GitHub - square/sudo_pair: Plugin for sudo that requires another human to approve and monitor privileged sudo sessions, accessed March 16, 2026,
https://github.com/square/sudo_pair
 23. sudo_plugin_python(8) - Linux manual page - man7.org, accessed March 16, 2026,
https://man7.org/linux/man-pages/man8/sudo_plugin_python.8.html
 24. Trying to tweak my sudo configuration - Ask Ubuntu, accessed March 16, 2026,
<https://askubuntu.com/questions/1382726/trying-to-tweak-my-sudo-configuration>
 25. sudo(8) - Linux manual page - man7.org, accessed March 16, 2026,
<https://man7.org/linux/man-pages/man8/sudo.8.html>
 26. Show HN: sudo_pair – dual control for sudo | Hacker News, accessed March 16, 2026,
<https://news.ycombinator.com/item?id=17015899>
 27. sudo_pair 0.9.0 released : r/rust - Reddit, accessed March 16, 2026,
https://www.reddit.com/r/rust/comments/8hppka/sudo_pair_090_released/
 28. sudo/plugins/python/example_approval_plugin.py at main - GitHub, accessed March 16, 2026,
https://github.com/sudo-project/sudo/blob/main/plugins/python/example_approval_plugin.py
 29. 7 sudo myths debunked | Opensource.com, accessed March 16, 2026,
<https://opensource.com/article/22/8/debunk-sudo-myths>
 30. Rust vs Go: Which One to Choose in 2025 - The JetBrains Blog, accessed March 16, 2026,
<https://blog.jetbrains.com/rust/2025/06/12/rust-vs-go/>
 31. Go vs. Rust: When to use Rust and when to use Go - LogRocket Blog, accessed March 16, 2026,
<https://blog.logrocket.com/go-vs-rust-when-use-rust-when-use-go/>
 32. Rust vs Go: The Ultimate Showdown for Backend Development - DEV Community, accessed March 16, 2026,
<https://dev.to/hamzakhan/rust-vs-go-the-ultimate-showdown-for-backend-development>

[lopment-5cll](#)

33. Go vs Rust: Which Programming Language Is Best For You? - YouTube, accessed March 16, 2026, <https://www.youtube.com/watch?v=bTQBlqG-l3w>
34. How to Implement Request-Reply Pattern in NATS - OneUptime, accessed March 16, 2026, <https://oneuptime.com/blog/post/2026-01-27-nats-request-reply-pattern/view>
35. Rust Daemon best practices - Stack Overflow, accessed March 16, 2026, <https://stackoverflow.com/questions/61443052/rust-daemon-best-practices>
36. Why is the select call not blocking on a unix domain socket? - Stack Overflow, accessed March 16, 2026, <https://stackoverflow.com/questions/24996336/why-is-the-select-call-not-blocking-on-a-unix-domain-socket>
37. Request-Reply (Python) - NATS by Example, accessed March 16, 2026, <https://natsbyexample.com/examples/messaging/request-reply/python>
38. Request-Reply - NATS Docs, accessed March 16, 2026, <https://docs.nats.io/nats-concepts/core-nats/repreply>
39. Rethinking Microservices: Using NATS to Dramatically Simplify Your Microservices, accessed March 16, 2026, <https://www.synadia.com/blog/rethinking-microservices>
40. Calling a sd_notify(0, "WATCHDOG=1") in a service - Stack Overflow, accessed March 16, 2026, <https://stackoverflow.com/questions/32947606/calling-a-sd-notify0-watchdog-1-in-a-service>
41. Notes on proper systemd services - Pascal's Scribbles, accessed March 16, 2026, <https://deterministic.space/writing-a-daemon.html>
42. Systemd Units Hardening - Rocky Linux Documentation, accessed March 16, 2026, https://docs.rockylinux.org/10/guides/security/systemd_hardening/
43. F40 Change Proposal: Systemd Security Hardening (System-Wide) - Fedora Discussion, accessed March 16, 2026, <https://discussion.fedoraproject.org/t/f40-change-proposal-systemd-security-hardening-system-wide/96423>
44. Using systemd features to secure services - Red Hat, accessed March 16, 2026, <https://www.redhat.com/en/blog/systemd-secure-services>
45. alegrey91/systemd-service-hardening - GitHub, accessed March 16, 2026, <https://github.com/alegrey91/systemd-service-hardening>
46. Human-in-the-Loop AI Agents: How to Design Approval Workflows for Safe and Scalable Automation - StackAI, accessed March 16, 2026, <https://www.stack-ai.com/insights/human-in-the-loop-ai-agents-how-to-design-approval-workflows-for-safe-and-scalable-automation>
47. Just in Time Access to PostgreSQL - Entitle, accessed March 16, 2026, <https://www.entitle.io/integrations/postgres>
48. Am I overcomplicating this database design for approvals [closed] - Stack Overflow, accessed March 16, 2026, <https://stackoverflow.com/questions/77174444/am-i-overcomplicating-this-database-design-for-approvals>

49. Create an audit trail for an Amazon Aurora PostgreSQL table | AWS Database Blog, accessed March 16, 2026,
<https://aws.amazon.com/blogs/database/create-an-audit-trail-for-an-amazon-aurora-postgresql-table/>
50. SSE vs WebSockets: Comparing Real-Time Communication Protocols - SoftwareMill, accessed March 16, 2026,
<https://softwaremill.com/sse-vs-websockets-comparing-real-time-communication-protocols/>
51. How to Use SSE vs WebSockets for Real-Time Communication - OneUptime, accessed March 16, 2026,
<https://oneuptime.com/blog/post/2026-01-27-sse-vs-websockets/view>
52. Real-Time Communication in Angular: SSE vs WebSocket - Aptuz Technology solutions, accessed March 16, 2026,
<https://aptuz.com/blog/a-dive-into-sse-and-web-sockets-in-angular/>
53. SSE or Websockets for push notifications? : r/node - Reddit, accessed March 16, 2026,
https://www.reddit.com/r/node/comments/tfct4e/sse_or_websockets_for_push_notifications/
54. WebSockets vs Server-Sent Events: Key differences and which to use in 2024 - Ably, accessed March 16, 2026, <https://ably.com/blog/websockets-vs-sse>
55. WebSocket vs Web Push vs Server-Sent Events: When to Use What? - DEV Community, accessed March 16, 2026,
<https://dev.to/ayushsrtv/websocket-vs-web-push-vs-server-sent-events-when-to-use-what-3214>
56. Angular 19 SSE using EventSource - Medium, accessed March 16, 2026,
<https://medium.com/@piyalidas.it/angular-19-sse-using-eventsource-ee770d18c7e4>
57. How to make a signal-based application in Angular 19, viable and what kind of design pattern? : r/Angular2 - Reddit, accessed March 16, 2026,
https://www.reddit.com/r/Angular2/comments/1o92qmb/how_to_make_a_signalbased_application_in_angular/
58. Signals • Overview • Angular, accessed March 16, 2026,
<https://angular.dev/guide/signals>
59. Reactivity with signals - Angular, accessed March 16, 2026,
<https://angular.dev/essentials/signals>
60. Deploying a workflow that waits for human approval in Step Functions - AWS Documentation, accessed March 16, 2026,
<https://docs.aws.amazon.com/step-functions/latest/dg/tutorial-human-approval.html>
61. Ubuntu: Sudo Critical Flaws - Local Privilege Escalation Threat - Linux Security, accessed March 16, 2026,
<https://linuxsecurity.com/news/security-vulnerabilities/sudo-flaws-linux-privilege-at-risk>
62. Abuse Elevation Control Mechanism: Sudo and Sudo Caching, Sub-technique T1548.003, accessed March 16, 2026,

- <https://attack.mitre.org/techniques/T1548/003/>
63. Package "sudo" (noble 24.04) - UbuntuUpdates, accessed March 16, 2026,
<https://www.ubuntuupdates.org/package/core/noble/main/security/sudo>
 64. Package "sudo" (noble 24.04) - Ubuntu Updates, accessed March 16, 2026,
<https://www.ubuntuupdates.org/package/core/noble/main/updates/sudo>
 65. Fixing Sudo vulnerability : r/Ubuntu - Reddit, accessed March 16, 2026,
https://www.reddit.com/r/Ubuntu/comments/1nzn6mh/fixing_sudo_vulnerability/
 66. Critical Sudo Vulnerabilities Let Local Users Gain Root Access on Linux, Impacting Major Distros - The Hacker News, accessed March 16, 2026,
<https://thehackernews.com/2025/07/critical-sudo-vulnerabilities-let-local.html>
 67. User management - Ubuntu Server documentation, accessed March 16, 2026,
<https://ubuntu.com/server/docs/how-to/security/user-management/>
 68. Adopting sudo-rs By Default in Ubuntu 25.10 - Project Discussion, accessed March 16, 2026,
<https://discourse.ubuntu.com/t/adopting-sudo-rs-by-default-in-ubuntu-25-10/60583>